

Orbital RAG: Ecossistema Espacial Inteligente

- **Integrantes:**
 - Arthur Guimarães Alentejo - rm564810,
 - Michael Araujo Rodrigues - rm565452,
 - Nathalia de Almeida Vasconcelos - rm563510
- **Frase Obrigatória:** **QUERO CONCORRER**
- **Github:** <https://github.com/Fiap-RJ/TIAO-2026/tree/main/2TIAO/Global-Solution-1>
- **Youtube:** https://youtu.be/HqM011QmoGs?si=zmJ_PISbHg-uX0xp

1. INTRODUÇÃO

A "Nova Economia Espacial" (*New Space*) transformou a exploração do cosmos numa indústria dinâmica, com o lançamento constante de satélites e monitorização de detritos orbitais. Este cenário gerou um volume massivo de dados técnicos fornecidos por agências como a NASA (através das APIs *NeoWs* e *DONKI*). Todavia, a fragmentação desses dados em múltiplas fontes torna a análise de risco lenta e complexa para operadores humanos.

O projeto **Orbital RAG** resolve este gargalo ao criar uma interface unificada e inteligente. Através da tecnologia de *Retrieval-Augmented Generation* (RAG), o sistema não só apresenta os dados brutos, como interpreta o contexto de riscos (como tempestades solares ou trajetórias de asteroides) em linguagem natural, agilizando a tomada de decisão no centro de comando espacial.

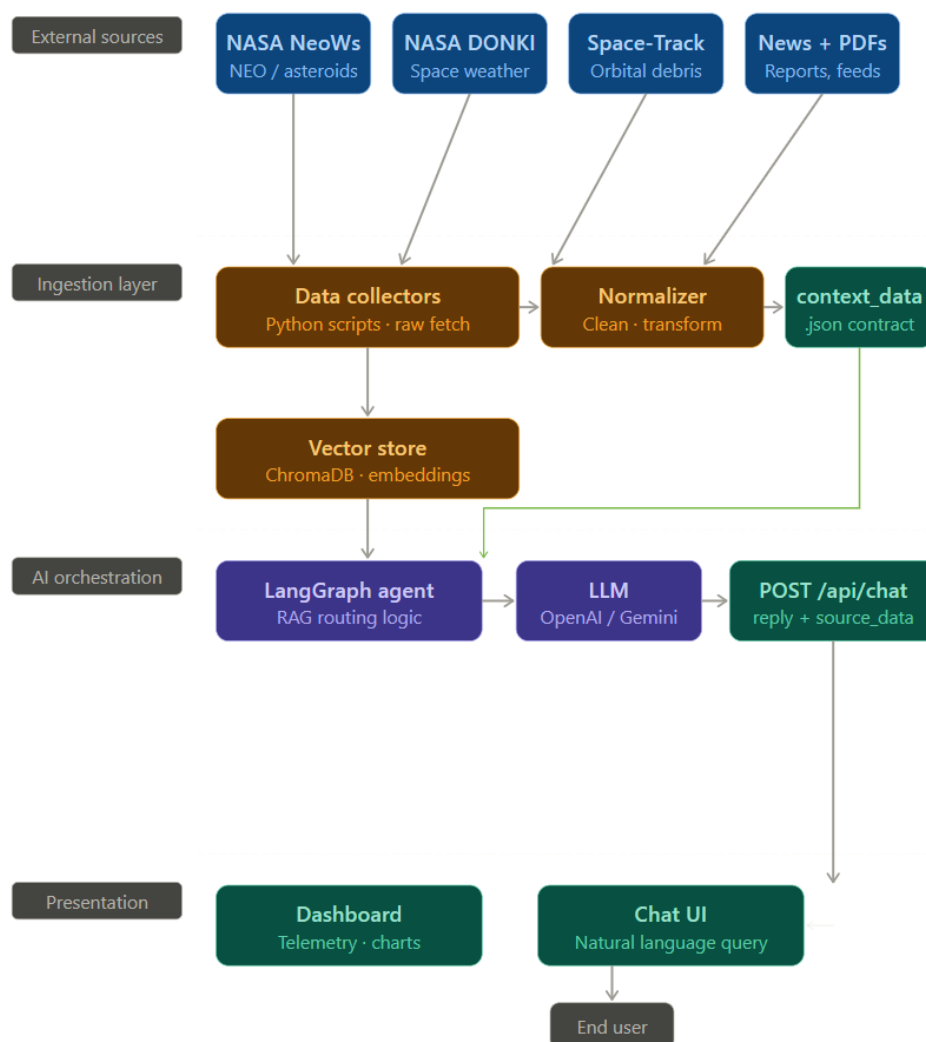
2. DESENVOLVIMENTO E ARQUITETURA

2.1 Visão da Arquitetura

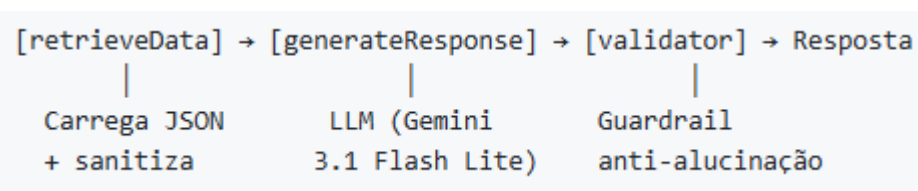
A arquitetura foi concebida seguindo o paradigma de microsserviços, garantindo desacoplamento entre a coleta de dados e a interface do usuário.

- **Camada de Ingestão (Data Pipeline):** Desenvolvida em Python, automatiza a extração de dados das APIs públicas da NASA. O processo limpa e normaliza os dados, garantindo que o motor de IA receba informações íntegras.
- **Motor de Inteligência (Backend):** Implementado em Node.js (NestJS), utiliza *LangGraph* para orquestração de agentes. O sistema decide dinamicamente quando consultar o contexto (base vetorial) ou processar dados estruturados em tempo real.
- **Camada de Apresentação (Frontend):** Construída em React + Vite, a interface foi desenhada com diretrizes de design inspiradas na NASA, focando em alto contraste, clareza e responsividade.

Figura 1 - Arquitetura do Orbital RAG



Pipeline RAG (LangGraph)



Stack Técnica

Camada	Tecnologia
Orquestração de Agentes	LangGraph (StateGraph) — @langchain/langgraph
LLM	Google Gemini 3.1 Flash Lite Preview
Backend / API	NestJS + TypeScript (Clean Architecture)
Engenharia de Dados	Python (requests, python-dotenv)
Containerização	Docker Compose
Gerenciamento de Deps (Data)	Poetry
Fontes de Dados	NASA NeoWs, NASA DONKI (CME, GST, SEP)

2.2 Decisões Técnicas

- **Escolha do LLM**
 - **Google Gemini 3.1 Flash Lite Preview:** Escolhido pelo custo-benefício para tarefas de síntese e interpretação de dados estruturados. Latência baixa, suporte nativo a português e custo zero na tier gratuita — ideal para um projeto acadêmico com múltiplas iterações.
- **Uso de RAG:** Escolhemos RAG em vez de apenas *Fine-Tuning* para garantir que o modelo utilize dados atualizados da NASA em tempo real, sem necessidade de re-treinamento constante.

Arquitetura RAG com LangGraph

O motor cognitivo é um **StateGraph** com 3 nós:

1. **retrieveData** — Carrega `context_data.json`, sanitiza e mapeia campos para entidades de domínio
 2. **generateResponse** — Constrói prompt determinístico (contexto + pergunta) e invoca o LLM
 3. **validator** — Guardrail anti-alucinação: verifica se a resposta referencia dados reais do JSON
Se a validação falha (alucinação detectada), o grafo retorna ao processamento via **aresta condicional**, garantindo resiliência.
- **Pipeline de Dados (Python):** A coleta de dados é desacoplada do backend. Scripts Python coletam dados de múltiplas APIs da NASA e normalizam para um formato unificado (`context_data.json`), que é consumido pelo backend via volume Docker.

Mapeamento NeoWs → JSON:

Campo NASA	Campo normalizado
<code>neo_reference_id</code>	<code>id_evento</code>
<code>name</code>	<code>nome</code>
<code>close_approach_data[0].close_approach_date</code>	<code>data_aproximacao</code>
<code>close_approach_data[0].miss_distance.kilometers</code>	<code>distancia_terra_km</code>
<code>is_potentially_hazardous_asteroid</code>	<code>risco_colisao</code>
<i>(gerado pelo normalizer)</i>	<code>resumo_alerta</code>

- **Stack Tecnológica:** A escolha de *React* para o front-end justifica-se pela sua capacidade de gerir estados dinâmicos (como a atualização de alertas em tempo real) de forma eficiente.

3. RESULTADOS ESPERADOS

A solução entrega um painel de controlo (Dashboard) capaz de converter dados técnicos espaciais em *insights* acionáveis.

- **Dashboard Dinâmico:** Visualização em tempo real de asteroides monitorizados e alertas de risco geomagnético.

Figura 2 - Dashboard do Orbital RAG com as principais informações

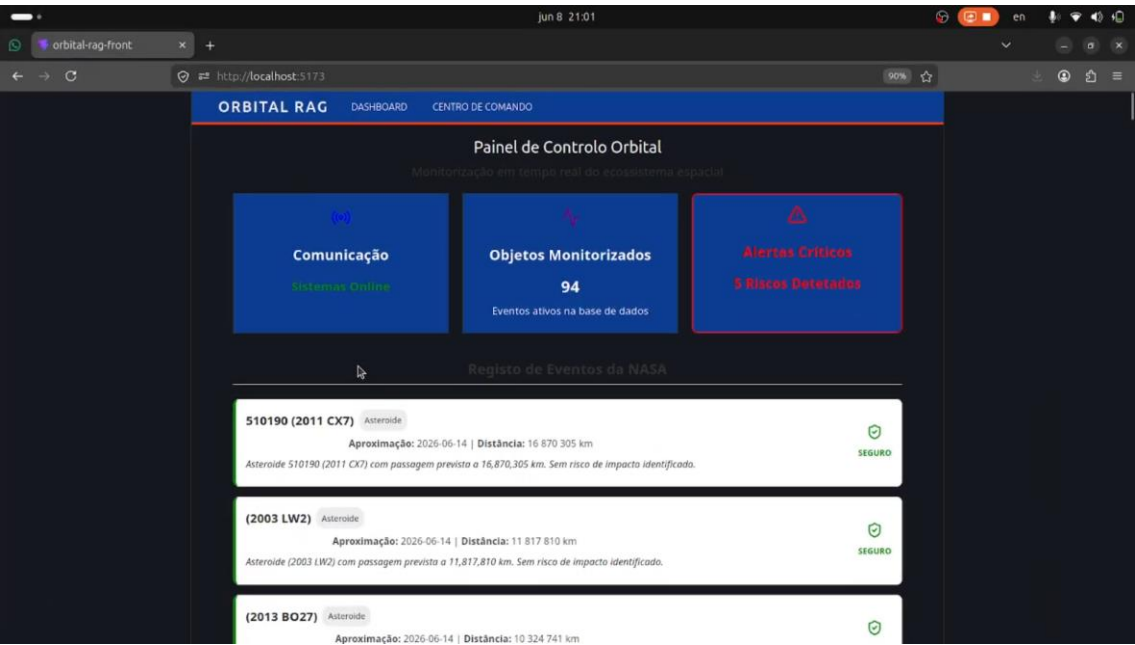
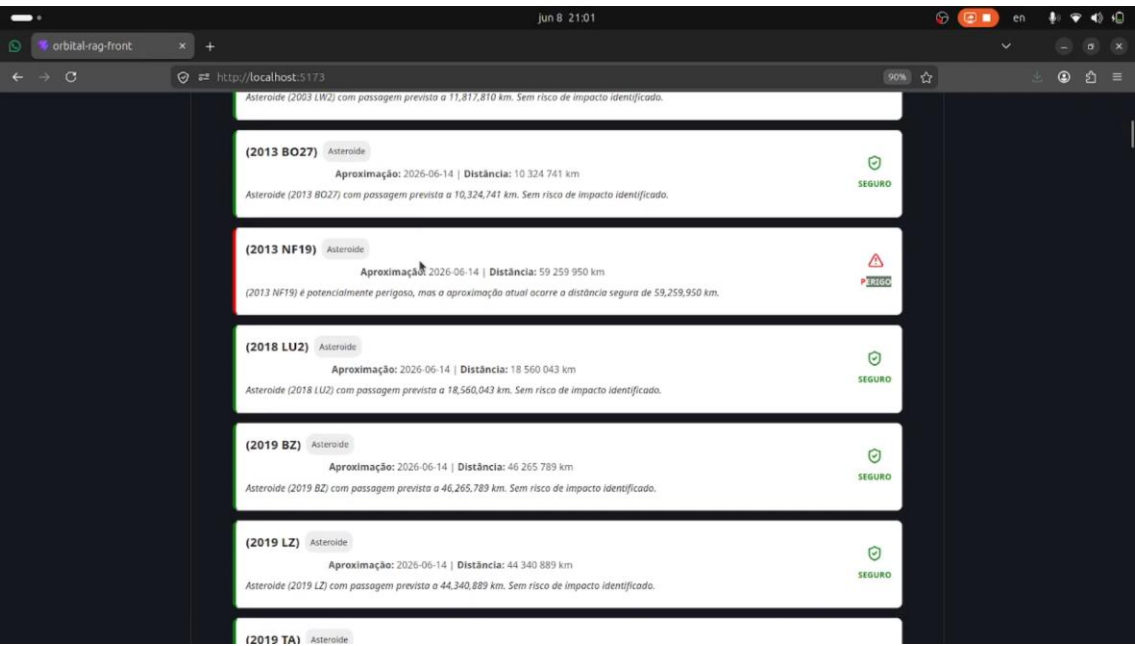


Figura 3 - Dashboard do Orbital RAG com os avisos



- **Chatbot Inteligente:** Capacidade de resposta contextualizada sobre eventos espaciais, com indicação das fontes consultadas (demonstrando transparência e responsabilidade algorítmica).

Figura 4 - Chatbot Comando Interativo com a pergunta e o aviso de comunicação

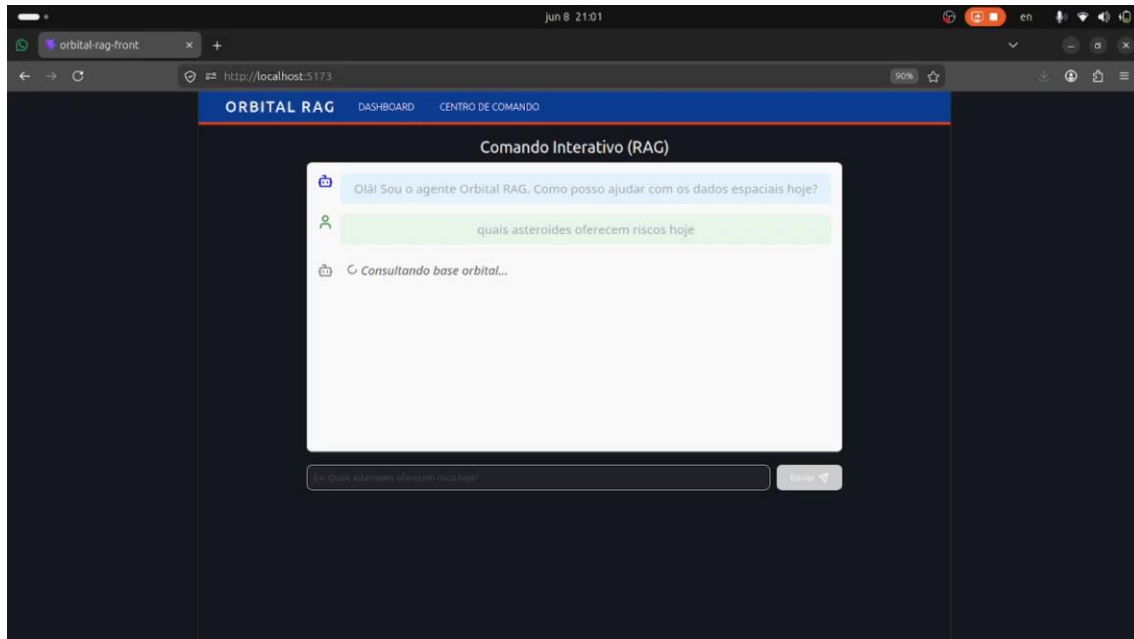
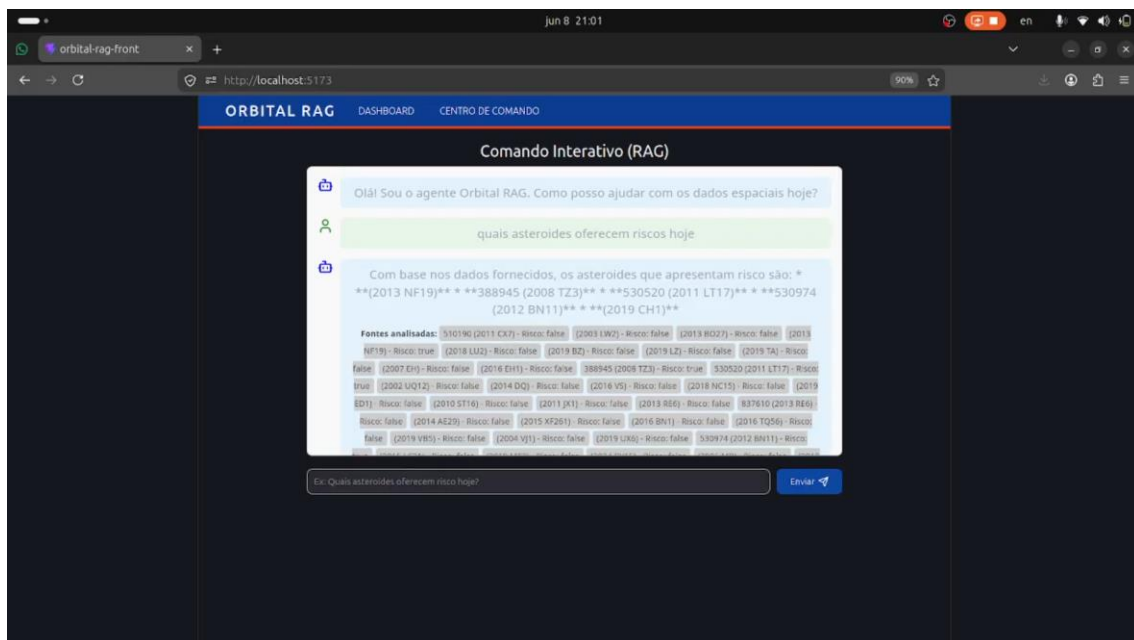


Figura 5 - Chatbot Comando Interativo com a resposta e fontes



4. CONCLUSÕES

O Orbital RAG demonstrou ser uma Prova de Conceito viável para a Nova Economia Espacial. Ao integrar automação (Python), arquitetura de back-end escalável (NestJS) e interfaces inteligentes (React), conseguimos mitigar o problema da dispersão de dados.

Como expansão futura, propõe-se a integração de sensores IoT reais (ESP32) para telemetria de hardware *in-loco* e técnicas de *Edge Computing* para análise de imagens espaciais diretamente na borda.

5. VÍDEO DE DEMONSTRAÇÃO

A demonstração funcional, abrangendo o Dashboard e a interação com o agente RAG, pode ser consultada no link abaixo:

https://youtu.be/HqM011QmoGs?si=zmJ_PISbHg-uX0xp